FedKG-Maint

Privacy-Preserving Predictive Maintenance with Federated Learning, a Knowledge Graph and Grounded LLM Explanations

Prepared by	Ahmed Ashraf Mohamed
Compute target	Google Colab free tier (NVIDIA T4)
Datasets	NASA C-MAPSS (FD001-FD004), Microsoft Azure Predictive Maintenance
Repository	github.com/<your-username>/FedKG-Maint

Summary

Machines fail rarely, so one factory seldom has enough failure examples to train a reliable prediction model, and factories cannot pool raw sensor data for privacy and business reasons. This project builds a framework in which several simulated factories jointly train one predictive maintenance model with federated learning, share only protected model updates, and give engineers clear explanations by combining the model output with a maintenance knowledge graph and a Large Language Model. The design is deliberately lightweight so that every experiment fits on a free Colab T4.

1. The Problem

Predictive maintenance uses sensor data to predict failures before they occur, cutting downtime and cost. In practice four obstacles appear:

- **Scarce failures.** A single factory has few failure events, so local models overfit or stay inaccurate.
- **No data sharing.** Raw telemetry and maintenance logs are confidential, which blocks centralized training.
- **Heterogeneity.** Factories differ in machines, operating conditions and fault modes (non-IID data), which hurts naive federated averaging and generalization to new sites.
- **Opaque predictions.** Engineers need to know why a failure is predicted and what to do. A bare RUL number is not actionable, and an ungrounded LLM may invent causes.

Research questions

- RQ1: Does federated learning close the gap between local-only and centralized training, and does it generalize to a held-out factory?
- RQ2: Which strategy copes best with heterogeneity: FedAvg or FedProx (or a personalized method)?
- RQ3: What accuracy do we lose at different privacy levels, and what can still leak?
- RQ4: Does knowledge-graph grounding make LLM explanations more faithful and less hallucinated?

2. Proposed Solution

The framework has four cooperating parts. Each factory keeps its data and builds a local knowledge graph on a shared schema. A federated learning loop trains the predictor, a privacy layer protects the updates, and an explanation module retrieves graph facts and prompts an LLM to explain each prediction.

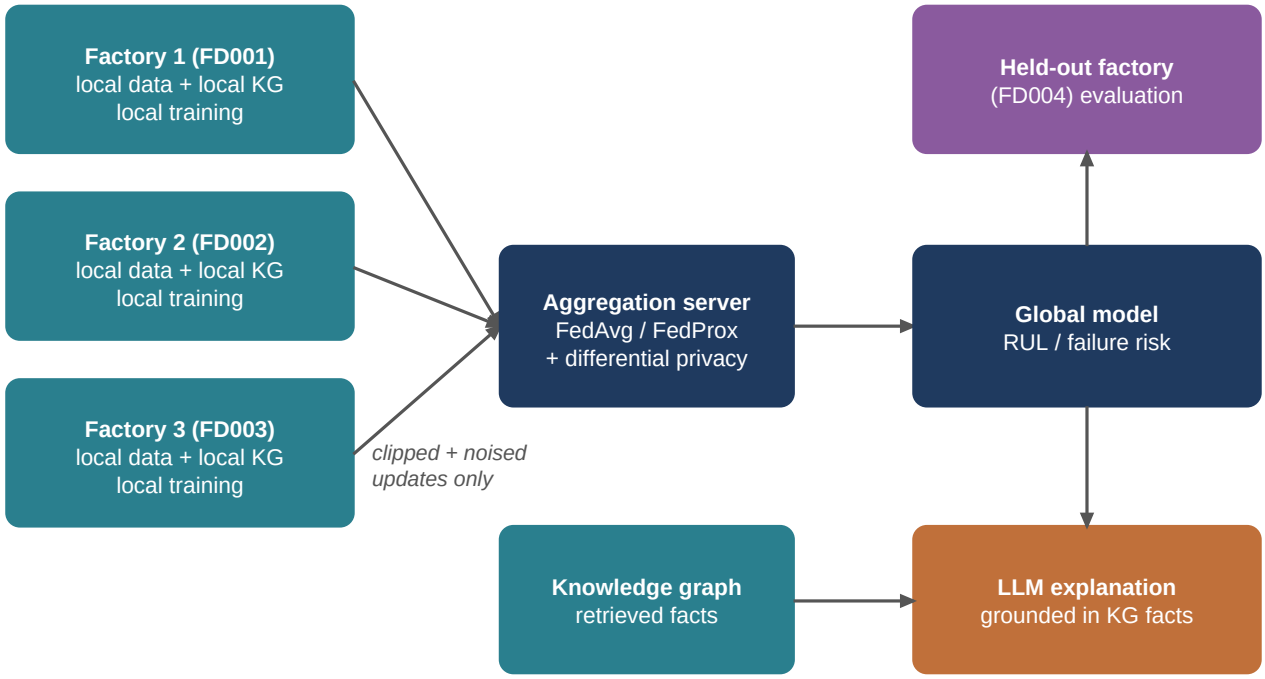

Figure 1. System overview. Raw data never leaves a factory; only clipped, noised model updates are sent.

3. Data and Federated Setup

Dataset	Content	Role in the project
NASA C-MAPSS	Turbofan run-to-failure telemetry, 4 sub-datasets (FD001-FD004) with different operating conditions and fault modes; tens of MB	RUL regression. FD001-FD003 are training factories, FD004 is the held-out factory
Azure Predictive Maintenance	Telemetry, errors, maintenance records, failures and machine metadata (5 CSV files); machines grouped by model	Failure within next N hours (classification), error codes and maintenance records feed the knowledge graph

Preprocessing.

- Use a fixed shared feature set (3 operating settings and 14 informative sensors) so all factories share one input size.
- Compute normalization statistics **locally** inside each factory from its own training data.
- Slide a 30-cycle window over each engine; target is RUL capped at 125 cycles (piecewise-linear degradation). Test uses the last window per engine.
- Azure: build lagged and rolling features per machine; label = failure within the next N hours; handle class imbalance with weighting.

4. Method

4.1 Predictive models

Primary model: 2-layer LSTM regressor (hidden size 64, ~50k parameters) for RUL. Optional upgrades: small Transformer encoder, and a GNN over the knowledge graph. Models are small by design: a full C-MAPSS training run takes minutes on a T4.

4.2 Federated learning

Simulated on one machine with one client per factory. Each round, clients start from the global weights, train locally, and return weight deltas; the server averages them weighted by dataset size. Strategies compared:

- **FedAvg** as the standard baseline.
- **FedProx**, which adds a proximal term (mu/2)||w - w_global||^2 to keep local models close to the global one under heterogeneity. A personalized method (e.g. local fine-tuning of the head) is an optional third.
- **Reference setups:** local-only training per factory (lower bound) and centralized training on pooled data (upper bound).
- All are evaluated on every factory's test set and, most importantly, on the held-out factory.

4.3 Privacy preservation

Client-level differential privacy on model updates: each update is clipped to L2 norm C, and Gaussian noise with standard deviation sigma*C/n is added to the average. We sweep the noise multiplier sigma, report the accuracy-privacy trade-off, and move to Opacus or an RDP accountant to quote a formal epsilon. The write-up also discusses residual leakage (membership and gradient-inversion style attacks on updates, and facts exposed by the shared graph schema).

4.4 Maintenance knowledge graph

A shared ontology links **Machine, Component, Sensor, ErrorCode, FailureMode and MaintenanceAction** through relations such as HAS_COMPONENT, MONITORED_BY, EXHIBITS, INDICATED_BY and RESOLVED_BY. Each factory instantiates its own local graph from its data (Azure

error and maintenance tables, C-MAPSS sensor-to-component mapping), optionally enriched with general domain knowledge. Implemented with *networkx*, so no graph database is needed.

4.5 LLM-based explanations

For each prediction the module (1) finds the relevant entities (sensors with abnormal trends, error codes, the predicted failure mode), (2) retrieves their graph neighbourhood as triples, and (3) prompts an LLM with the prediction plus *only* those facts, instructing it not to add anything else. To fit a T4 we use a small quantized open model (1-3B parameters) or a free-tier API. We compare grounded versus ungrounded explanations.

5. Evaluation Plan

Aspect	Metrics / procedure
Prediction (C-MAPSS)	RMSE and the asymmetric NASA score on each factory test set and the held-out factory
Prediction (Azure)	F1-score and PR-AUC (failure within next N hours)
Federated behaviour	Communication cost (bytes per round x rounds); effect of number of clients and degree of heterogeneity
Privacy	Performance versus noise multiplier / epsilon; discussion of leakage
Explanations	Faithfulness (every claim traceable to model output or KG facts), factual correctness, hallucination rate; small human rating if possible
Ablations	No KG, no privacy, no LLM, FedAvg vs FedProx; 5 seeds per setting with mean +/- std and a significance test (paired t-test or Wilcoxon)

6. Why this fits a free Colab T4

- Datasets are small (C-MAPSS in tens of MB, Azure in five small CSV files); no download or memory problems.
- Models are small, so one training run takes minutes and 5-seed experiments are cheap.
- Federated learning is simulated in a single process: no multi-machine setup.
- The LLM runs quantized or through an API, and the knowledge graph is plain Python, so the T4's 16 GB is enough.
- Results and checkpoints are saved to Google Drive after each experiment to survive session disconnects.

7. Timeline

Phase	Work	Output
1	C-MAPSS preprocessing, centralized and local baselines	Baseline table
2	FedAvg and FedProx, held-out factory, clients and heterogeneity study	Federated results
3	Differential privacy sweep, leakage discussion	Privacy-utility curve
4	Knowledge graph and LLM explanations (grounded vs ungrounded)	Explanation evaluation
5	Azure PdM failure prediction (F1, PR-AUC)	Second-dataset results
6	Ablations, multiple seeds, significance tests, final report	Report and repo release

8. Risks and Mitigation

Risk	Mitigation
Many components (FL, DP, KG, LLM) to deliver	Build in phases; each phase produces a usable result and the project is valid even if later phases are reduced
FedAvg diverges on non-IID clients	FedProx, lower learning rate, more local epochs tuning, per-client normalization
DP noise destroys accuracy	Tune clipping norm; report the trade-off honestly, it is a finding
LLM hallucination	Strict grounding prompt, factual checks against retrieved triples, report hallucination rate
Colab disconnects	Short runs, checkpoints and results saved to Drive

9. Repository Structure

Path	Contents
src/fedkg/	data.py, models.py, federated.py, privacy.py, kg.py, explain.py, metrics.py
scripts/	train_federated.py (modes: federated, local, centralized)
configs/	default.yaml (data, model, federated and privacy settings)
notebooks/	Colab notebooks for running experiments
results/, docs/	Metrics and plots; this proposal